

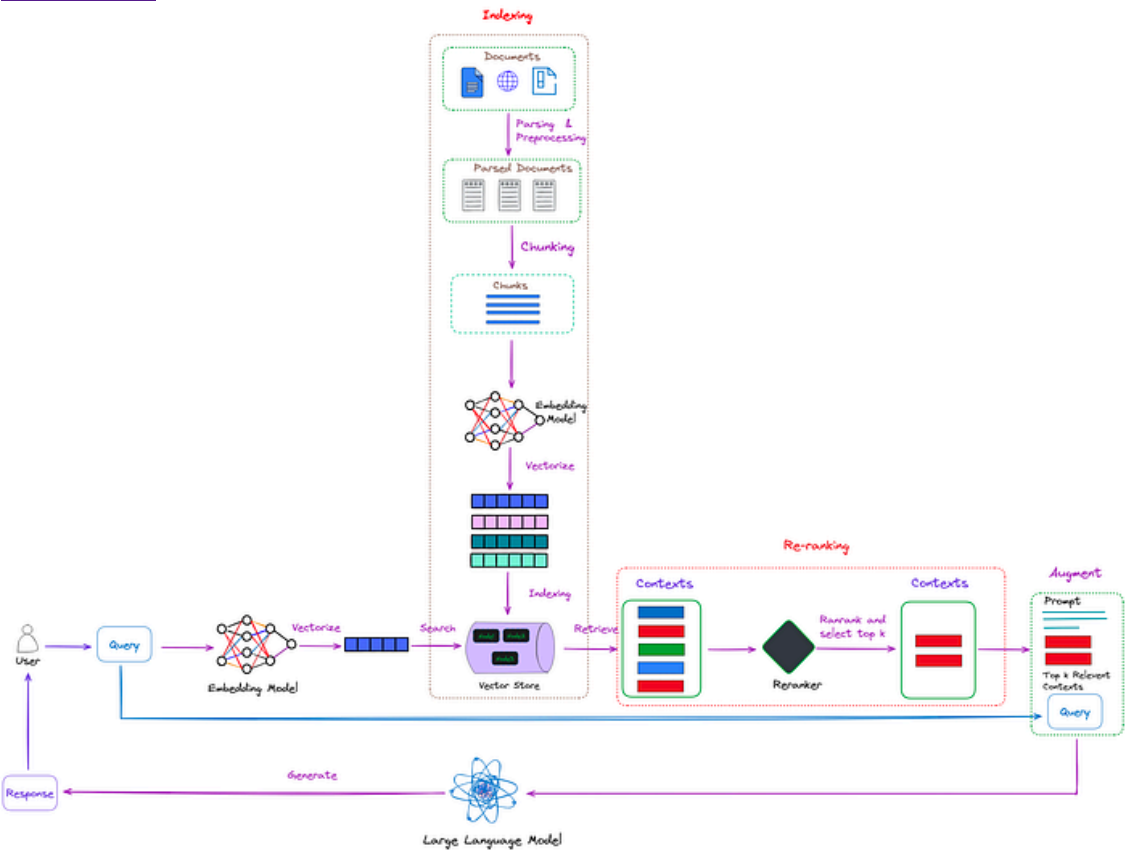
我的学习笔记

土猛的员外

- [主页](#)
- [归档](#)
- [标签](#)
- [订阅](#)

【翻译】高级RAG 04：重排（Rerank）

2024-02-22



Re-ranking在检索增广生成(RAG)过程中起着至关重要的作用。在朴素的RAG方法中，可以检索大量上下文，但并非所有上下文都与问题相关。重新排序允许对文档进行重新排序和过滤，将相关的文档放在最前面，从而提高RAG的有效性。

本文介绍了RAG的重新排序技术，并演示了如何使用两种方法合并重新排序功能

介绍Re-ranking

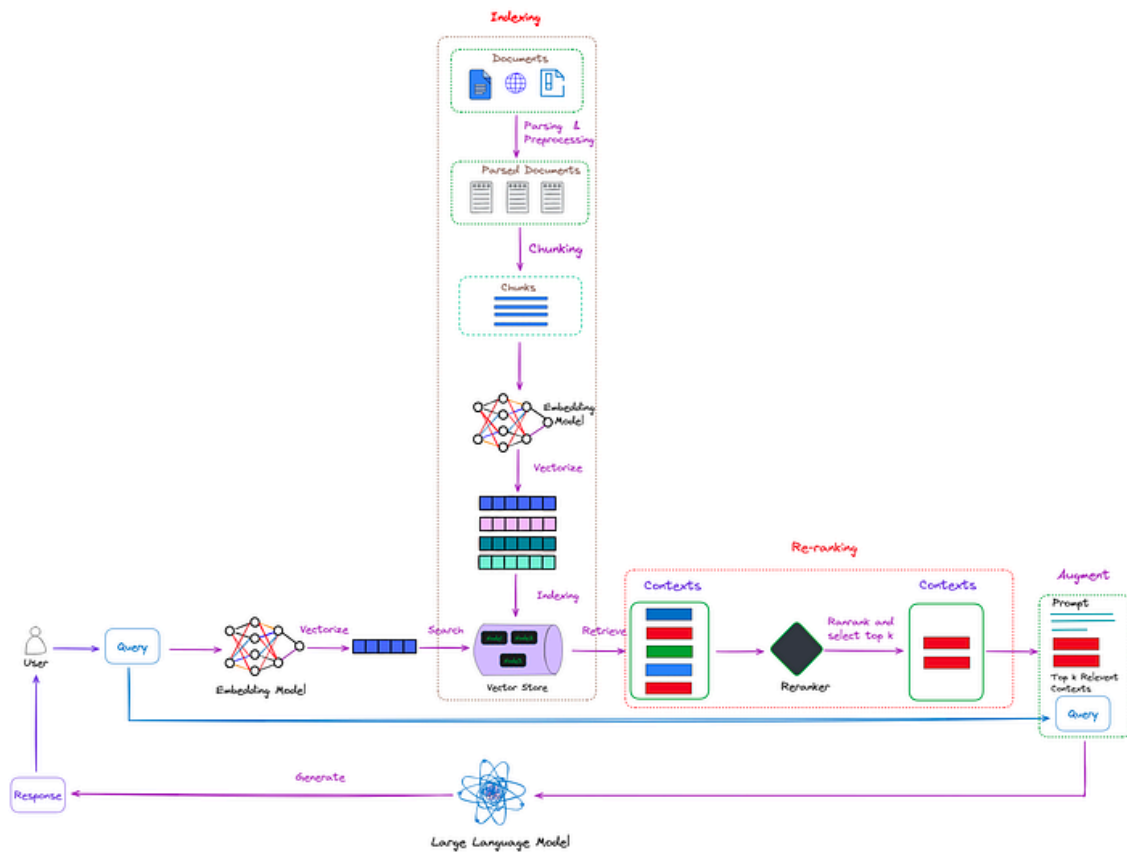


图1：在RAG中重排，重排的任务是评估这些上下文的相关性，并优先考虑最有可能提供准确和相关答案的上下文（红框）。图片来自作者。

如图1所示，重排的任务就像一个智能过滤器。当检索器从索引集合中检索多个上下文时，这些上下文可能与用户的查询具有不同的相关性。有些上下文可能非常相关(在图1中的红色框中突出显示)，而其他上下文可能只是稍微相关，甚至不相关(在图1中的绿色和蓝色框中突出显示)。

重排的任务是评估这些上下文的相关性，并优先考虑最有可能提供准确和相关答案的上下文。这允许LLM在生成答案时优先考虑这些排名靠前的上下文，从而提高响应的准确性和质量。

简单来说，重新排名就像在开卷考试中帮助你从一堆学习材料中选择最相关的参考资料，这样你就可以更有效、更准确地回答问题。

本文介绍的重新排序方法主要分为以下两种：

- 重新排序模型:这些模型考虑文档和查询之间的交互特征，以更准确地评估它们的相关性。
- LLM：LLM的出现为重新排名开辟了新的可能性。通过彻底理解整个文档和查询，可以更全面地捕获语义信息。

使用Rerank模型

与Embedding模型不同，重排序模型以查询和上下文作为输入，直接输出相似度分数，而不是Embeddings。重要的是要注意，重新排序模型是使用交叉熵损失函数进行优化的，允许不限于特定范围的相关性分数，甚至可以是负的。

目前，没有很多可用的重排模型。一种选择是Cohere提供的在线模型，可以通过API访问。还有一些开源模型，如big-reranker-base和big-reranker-large等。

图2显示了使用命中率和平均倒数排名(MRR)指标的评估结果：

Embedding	WithoutReranker		bge-reranker-base		bge-reranker-large		Cohere-Reranker	
	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR
OpenAI	0.876404	0.718165	0.91573	0.832584	0.910112	0.855805	0.926966	0.86573
bge-large	0.752809	0.597191	0.859551	0.805243	0.865169	0.816011	0.876404	0.822753
llm-embedder	0.814607	0.587266	0.870787	0.80309	0.876404	0.824625	0.882022	0.830243
Cohere-v2	0.780899	0.570506	0.876404	0.798127	0.876404	0.825281	0.876404	0.815543
Cohere-v3	0.825843	0.624532	0.882022	0.806086	0.882022	0.834644	0.88764	0.836049
Voyage	0.831461	0.68736	0.926966	0.837172	0.91573	0.858614	0.91573	0.851217
JinaAI-Small	0.831461	0.614045	0.91573	0.843071	0.926966	0.857303	0.926966	0.868633
JinaAI-Base	0.848315	0.68221	0.938202	0.846348	0.938202	0.868539	0.932584	0.873689
Google-PaLM	0.865169	0.719476	0.910112	0.833708	0.910112	0.85309	0.910112	0.855712

图2：使用命中率和平均倒数排名(MRR)指标的评估结果。来源:[提升RAG:选择最佳Embedding和重新排名模型](#)

从这个评价结果，我们可以看到：

- 无论使用何种Embedding模型，重排都具有更高的命中率和MRR，表明重新排名的影响显著。
- 目前，最好的重新排名模式是Cohere，但它是一个付费服务。开源big-rerrank-large模型具有与cohere类似的功能。Embedding模型和重排模型的组合也会产生影响，所以开发者可能需要在实际过程中尝试不同的组合。

在本文中，将使用基于更大rank的模型。

环境配置

导入相关库，设置环境变量和全局变量

```
1 import os
2 os.environ["OPENAI_API_KEY"] = "YOUR_OPENAI_KEY"
3
4 from llama_index import VectorStoreIndex, SimpleDirectoryReader
5 from llama_index.postprocessor.flag_embedding_reranker import FlagEmbeddingReranker
6 from llama_index.schema import QueryBundle
7
8 dir_path = "YOUR_DIR_PATH"
```

目录中只有一个PDF文件，使用的是论文“[TinyLlama:一个开源的小型语言模型](#)”。

```
1 (py) Florian:~ Florian$ ls /Users/Florian/Downloads/pdf_test/
2 tinyllama.pdf
```

使用LlamaIndex构建一个简单的检索器

```
1 documents = SimpleDirectoryReader(dir_path).load_data()
2 index = VectorStoreIndex.from_documents(documents)
3 retriever = index.as_retriever(similarity_top_k = 3)
```

基本检索

```
1 query = "Can you provide a concise description of the TinyLlama model?"
2 nodes = retriever.retrieve(query)
3 for node in nodes:
4     print('-----')
5     display_source_node(node, source_length = 500)
```

display_source_node函数改编自[llama_index源代码](#)。原来的功能是为Jupyter notebook设计的，所以做了如下修改：

```
1 from llama_index.schema import ImageNode, MetadataMode, NodeWithScore
2 from llama_index.utils import truncate_text
3
4 def display_source_node(
5     source_node: NodeWithScore,
6     source_length: int = 100,
7     show_source_metadata: bool = False,
8     metadata_mode: MetadataMode = MetadataMode.NONE,
```

```

9 ) -> None:
10     """Display source node"""
11     source_text_fmt = truncate_text(
12         source_node.node.get_content(metadata_mode=metadata_mode).strip(), source_length
13     )
14     text_md = (
15         f"Node ID: {source_node.node.node_id} \n"
16         f"Score: {source_node.score} \n"
17         f"Text: {source_text_fmt} \n"
18     )
19     if show_source_metadata:
20         text_md += f"Metadata: {source_node.node.metadata} \n"
21     if isinstance(source_node.node, ImageNode):
22         text_md += "Image:"
23
24     print(text_md)
25     # display(Markdown(text_md))
26     # if isinstance(source_node.node, ImageNode) and source_node.node.image is not None:
27     #     display_image(source_node.node.image)

```

基本检索结果如下，表示重排前的前3个节点:

```

1 -----
2 Node ID: 438b9d91-cd5a-44a8-939e-3ecd77648662
3 Score: 0.8706055408845863
4 Text: 4 Conclusion
5 In this paper, we introduce TinyLlama, an open-source, small-scale language model. To promote
6 transparency in the open-source LLM pre-training community, we have released all relevant infor-
7 mation, including our pre-training code, all intermediate model checkpoints, and the details of our
8 data processing steps. With its compact architecture and promising performance, TinyLlama can
9 enable end-user applications on mobile devices, and serve as a lightweight platform for testing a
10 w...
11
12 -----
13 Node ID: ca4db90f-5c6e-47d5-a544-05a9a1d09bc6
14 Score: 0.8624531691777889
15 Text: TinyLlama: An Open-Source Small Language Model
16 Peiyuan Zhang*Guangtao Zeng*Tianduo Wang Wei Lu
17 StatNLP Research Group
18 Singapore University of Technology and Design
19 {peiyuan_zhang, tianduo_wang, @sutd.edu.sg">luwei}@sutd.edu.sg
20 guangtao_zeng@mymail.sutd.edu.sg
21 Abstract
22 We present TinyLlama, a compact 1.1B language model pretrained on around 1
23 trillion tokens for approximately 3 epochs. Building on the architecture and tok-
24 enizer of Llama 2 (Touvron et al., 2023b), TinyLlama leverages various advances
25 contr...
26
27 -----
28 Node ID: e2d97411-8dc0-40a3-9539-a860d1741d4f
29 Score: 0.8346160605298356
30 Text: Although these works show a clear preference on large models, the potential of training smaller
31 models with larger dataset remains under-explored. Instead of training compute-optimal language
32 models, Touvron et al. (2023a) highlight the importance of the inference budget, instead of focusing
33 solely on training compute-optimal language models. Inference-optimal language models aim for
34 optimal performance within specific inference constraints This is achieved by training models with
35 more tokens...

```

Re-ranking

要对上述节点重排，请使用基于bge-reranker-base模型。

```

1 print('-----')
2 print('Start reranking...')
3
4 reranker = FlagEmbeddingReranker(
5     top_n = 3,
6     model = "BAAI/bge-reranker-base",
7 )
8
9 query_bundle = QueryBundle(query_str=query)
10 ranked_nodes = reranker._postprocess_nodes(nodes, query_bundle = query_bundle)
11 for ranked_node in ranked_nodes:
12     print('-----')
13     display_source_node(ranked_node, source_length = 500)

```

重排后的结果如下:

```

1 -----
2 Start reranking...
3 -----
4 Node ID: ca4db90f-5c6e-47d5-a544-05a9a1d09bc6
5 Score: -1.584416151046753
6 Text: TinyLlama: An Open-Source Small Language Model
7 Peiyuan Zhang*Guangtao Zeng*Tianduo Wang Wei Lu
8 StatNLP Research Group
9 Singapore University of Technology and Design
10 {peiyuan_zhang, tianduo_wang, @sutd.edu.sg">luwei}@sutd.edu.sg
11 guangtao_zeng@mymail.sutd.edu.sg
12 Abstract
13 We present TinyLlama, a compact 1.1B language model pretrained on around 1
14 trillion tokens for approximately 3 epochs. Building on the architecture and tok-
15 enizer of Llama 2 (Touvron et al., 2023b), TinyLlama leverages various advances
16 contr...
17
18 -----
19 Node ID: e2d97411-8dc0-40a3-9539-a860d1741d4f
20 Score: -1.7028117179870605
21 Text: Although these works show a clear preference on large models, the potential of training smaller
22 models with larger dataset remains under-explored. Instead of training compute-optimal language
23 models, Touvron et al. (2023a) highlight the importance of the inference budget, instead of focusing
24 solely on training compute-optimal language models. Inference-optimal language models aim for
25 optimal performance within specific inference constraints This is achieved by training models with
26 more tokens...
27
28 -----
29 Node ID: 438b9d91-cd5a-44a8-939e-3ecd77648662
30 Score: -2.904750347137451
31 Text: 4 Conclusion
32 In this paper, we introduce TinyLlama, an open-source, small-scale language model. To promote
33 transparency in the open-source LLM pre-training community, we have released all relevant infor-
34 mation, including our pre-training code, all intermediate model checkpoints, and the details of our
35 data processing steps. With its compact architecture and promising performance, TinyLlama can
36 enable end-user applications on mobile devices, and serve as a lightweight platform for testing a
37 w...

```

可以看出，重新排序后，ID为ca4db90f-5c6e-47d5-a544-05a9a1d09bc6的节点的排序由2变为1。这意味着最相关的上下文被排在第一位。

使用LLM作为ranker

现有涉及LLM的重排序方法大致可以分为三类：带重排任务的微调LLM、使用提示工程的LLM重排，以及在训练过程中使用LLM进行数据增强。

使用提示工程的LLM重排方法成本较低。下面是使用[RankGPT](#)的演示，它已[集成到LlamaIndex](#)中。

RankGPT的思想是使用LLM（如ChatGPT或GPT-4或其他大语言模型）执行zero-shot顺序通道重新排名。它采用排列生成方法和滑动窗口策略来有效地重新排列通道。

如图3所示，[论文](#)提出了三种可行的方法。

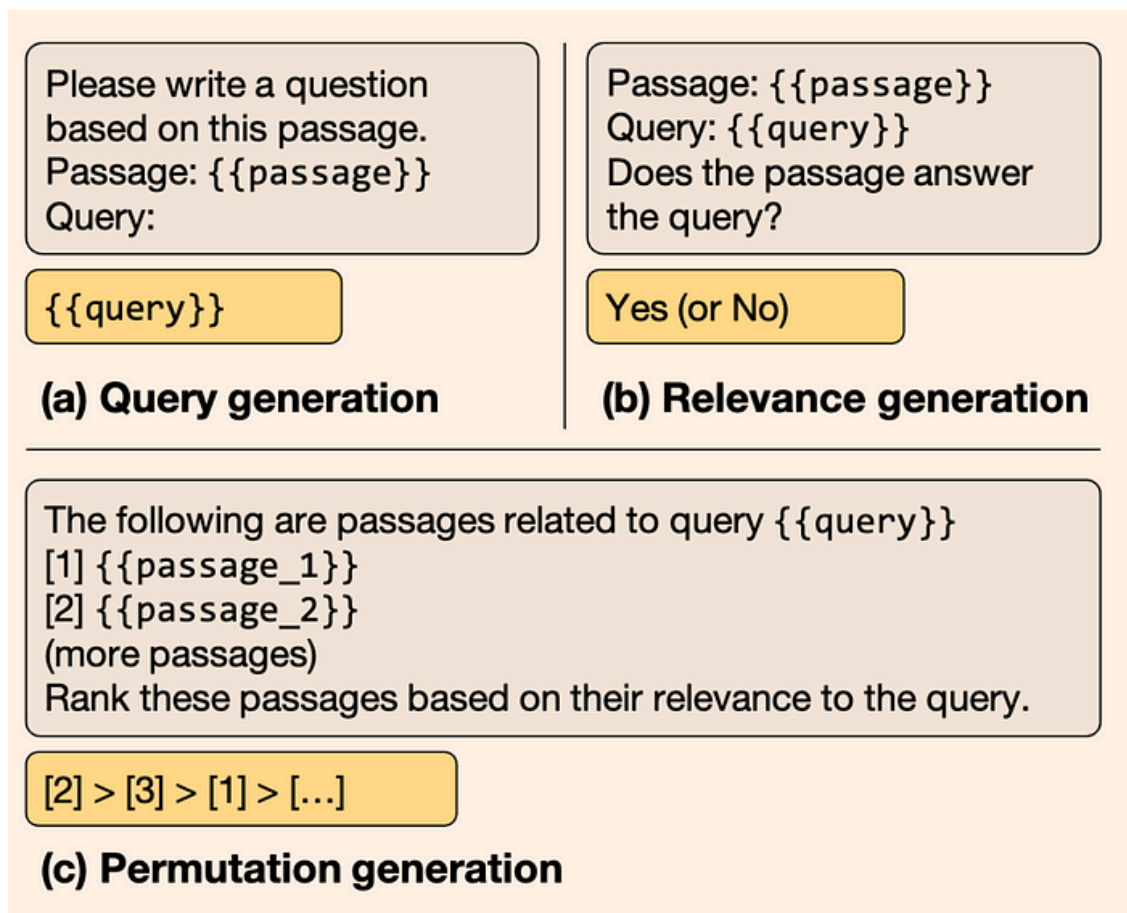


图3：使用大语言模型对zero-shot通道重新排序的三种指令。灰色和黄色块表示模型的输入和输出。(a)查询生成依赖于大语言模型的日志概率来生成基于段落的查询。(b)关联生成指导大语言模型输出关联判断。(c)置换生成生成一组段落的排序列表。来源：<https://arxiv.org/pdf/2304.09542.pdf>

前两种方法是传统的方法，其中给每个文档一个分数，然后根据这个分数对所有段落进行排序。

本文提出了第三种方法——置换生成法。具体来说，该模型不依赖于外部评分，而是直接对段落进行端到端排序。也就是说，直接利用LLM的语义理解能力，对所有候选段落进行相关性排序。

然而，通常候选文档的数量非常大，而LLM的输入是有限的。因此，一次输入所有文本通常是不可能的。

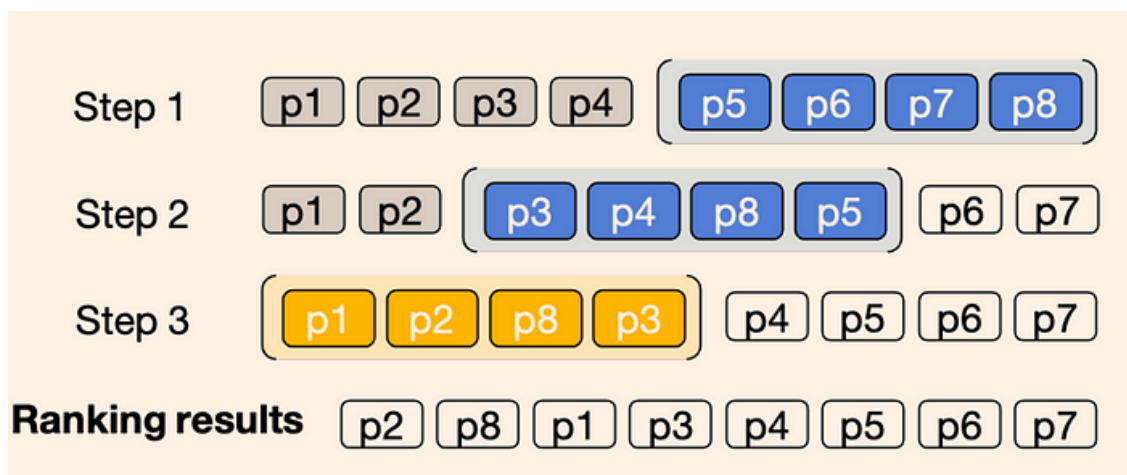


图4：使用窗口大小为4、步长为2的滑动窗口对8个通道重新排序的示意图。蓝色表示前两个窗口，而黄色表示最后一个窗口。滑动窗口以“从后到先”的顺序应用，这意味着前一个窗口中的前两个段落将参与下一个窗口的重新排序。来源：<https://arxiv.org/pdf/2304.09542.pdf>

因此，如图4所示，引入了一种滑动窗口方法，它遵循冒泡排序的思想。每次只对第一个**4** 文本进行排序，然后移动窗口，对随后的**4** 文本进行排序。通过对整个文本进行迭代，我们可以得到性能最好的top文本。

请注意，为了使用RankGPT，您需要安装较新版本的LlamaIndex。我之前安装的版本(0.9.29)不包括RankGPT所需的代码。因此，我使用LlamaIndex版本0.9.45.post1创建了一个新的conda环境。

```

1 from llama_index.postprocessor import RankGPTRerank
2 from llama_index.llms import OpenAI
3 reranker = RankGPTRerank(
4     top_n = 3,
5     llm = OpenAI(model="gpt-3.5-turbo-16k"),
6     # verbose=True,
7 )

```

总体结果如下:

```

1 (llamaindex_new) Florian:~ Florian$ python /Users/Florian/Documents/rerank.py
2 -----
3 Node ID: 20de8234-a668-442d-8495-d39b156b44bb
4 Score: 0.8703492815379594
5 Text: 4 Conclusion
6 In this paper, we introduce TinyLlama, an open-source, small-scale language model. To promote
7 transparency in the open-source LLM pre-training community, we have released all relevant infor-
8 mation, including our pre-training code, all intermediate model checkpoints, and the details of our
9 data processing steps. With its compact architecture and promising performance, TinyLlama can
10 enable end-user applications on mobile devices, and serve as a lightweight platform for testing a
11 w...
12
13 -----
14 Node ID: 47ba3955-c6f8-4f28-a3db-f3222b3a09cd
15 Score: 0.8621633467539512
16 Text: TinyLlama: An Open-Source Small Language Model
17 Peiyuan Zhang*Guangtao Zeng*Tianduo Wang Wei Lu
18 StatNLP Research Group
19 Singapore University of Technology and Design
20 {peiyuan_zhang, tianduo_wang, @sutd.edu.sg">luwei}@sutd.edu.sg
21 guangtao_zeng@mymail.sutd.edu.sg
22 Abstract
23 We present TinyLlama, a compact 1.1B language model pretrained on around 1
24 trillion tokens for approximately 3 epochs. Building on the architecture and tok-
25 enizer of Llama 2 (Touvron et al., 2023b), TinyLlama leverages various advances
26 contr...
27
28 -----
29 Node ID: 17cd9896-473c-47e0-8419-16b4ac615a59
30 Score: 0.8343984516104476
31 Text: Although these works show a clear preference on large models, the potential of training smaller
32 models with larger dataset remains under-explored. Instead of training compute-optimal language
33 models, Touvron et al. (2023a) highlight the importance of the inference budget, instead of focusing
34 solely on training compute-optimal language models. Inference-optimal language models aim for
35 optimal performance within specific inference constraints This is achieved by training models with
36 more tokens...
37
38 -----
39 Start reranking...
40 -----
41 Node ID: 47ba3955-c6f8-4f28-a3db-f3222b3a09cd
42 Score: 0.8621633467539512
43 Text: TinyLlama: An Open-Source Small Language Model
44 Peiyuan Zhang*Guangtao Zeng*Tianduo Wang Wei Lu
45 StatNLP Research Group
46 Singapore University of Technology and Design
47 {peiyuan_zhang, tianduo_wang, @sutd.edu.sg">luwei}@sutd.edu.sg
48 guangtao_zeng@mymail.sutd.edu.sg
49 Abstract
50 We present TinyLlama, a compact 1.1B language model pretrained on around 1
51 trillion tokens for approximately 3 epochs. Building on the architecture and tok-
52 enizer of Llama 2 (Touvron et al., 2023b), TinyLlama leverages various advances
53 contr...
54
55 -----
56 Node ID: 17cd9896-473c-47e0-8419-16b4ac615a59
57 Score: 0.8343984516104476
58 Text: Although these works show a clear preference on large models, the potential of training smaller
59 models with larger dataset remains under-explored. Instead of training compute-optimal language
60 models, Touvron et al. (2023a) highlight the importance of the inference budget, instead of focusing
61 solely on training compute-optimal language models. Inference-optimal language models aim for
62 optimal performance within specific inference constraints This is achieved by training models with
63 more tokens...
64
65 -----
66 Node ID: 20de8234-a668-442d-8495-d39b156b44bb
67 Score: 0.8703492815379594
68 Text: 4 Conclusion
69 In this paper, we introduce TinyLlama, an open-source, small-scale language model. To promote
70 transparency in the open-source LLM pre-training community, we have released all relevant infor-
71 mation, including our pre-training code, all intermediate model checkpoints, and the details of our
72 data processing steps. With its compact architecture and promising performance, TinyLlama can

```

73 enable end-user applications on mobile devices, and serve as a lightweight platform for testing a
74 w...

请注意，由于使用LLM，重新排名后的分数没有更新。当然，这不是至关重要的。

从结果中可以看出，重新排序后，排名前1的结果是包含答案的正确文本，这与之前使用重新排序模型得到的结果一致。

评估

我们可以修改第三节的代码如下：

```
1 reranker = FlagEmbeddingReranker(  
2     top_n = 3,  
3     model = "BAAI/bge-reranker-base",  
4     use_fp16 = False  
5 )  
6  
7 # or using LLM as reranker  
8 # from llama_index.postprocessor import RankGPTReRank  
9 # from llama_index.llms import OpenAI  
10 # reranker = RankGPTReRank(  
11 #     top_n = 3,  
12 #     llm = OpenAI(model="gpt-3.5-turbo-16k"),  
13 #     # verbose=True,  
14 # )  
15  
16 query_engine = index.as_query_engine(      # add reranker to query_engine  
17     similarity_top_k = 3,  
18     node_postprocessors=[reranker]  
19 )  
20 # query_engine = index.as_query_engine()    # original query_engine
```

感兴趣的读者可以试用一下。

结论

总体而言，本文介绍了重新排序的原理和两种主流方法。

其中，使用重排序模型的方法是轻量级的，开销较小。

另一方面，使用LLM的方法在多个基准测试中表现良好但成本较高，并且仅在使用ChatGPT和GPT-4时表现良好，而在使用FLAN-T5和Vicuna-13B等其他开源模型时表现不佳。

因此，在实际项目中，需要进行特定的权衡。

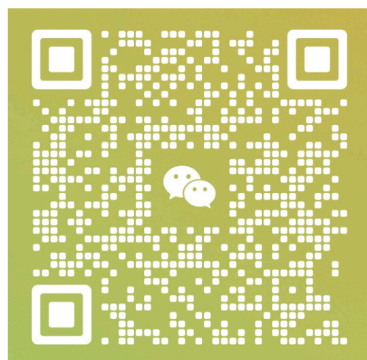
英文原文：<https://pub.towardsai.net/advanced-rag-04-re-ranking-85f6ae8170b1rag>

TorchV AI支持试用！

如您有大模型应用方面的企业需求，欢迎咨询！



最新文章, 请关注“土猛的员外” 公众号



作者的微信二维码

TOR
CHV

[首页](#) [产品](#) [价格](#) [文档](#) [公司](#)

让您的业务, 快速具备 AI能力

使用TorchV AI 可以快速与各类非结构化数据、结构化数据建立对话, 让数据在业务中发挥更大价值!

[开始了解](#)

结构化数据

非结构化数据
通常无法用于业务系统, 价值无法被挖掘

企业数据

TOR
CHV

APP